小云宝 OV2640 实时画面问题复盘与稳定方案

ESP32-S3 N16R8 + OV2640（板载 Y1）+ ST7789

小云宝项目

2026-09-18

- 1. 最终结论
- 2. 最终硬件配置
  - 2.1 摄像头引脚
  - 2.2 ST7789 引脚
- 3. 已取得的关键证据
  - 3.1 RGB565 独立抓帧通过
  - 3.2 纯串口 JPEG 连续抓帧通过
  - 3.3 加入 ST7789 后仍通过
  - 3.4 移回自制 PCB 后多次通过
- 4. 故障演变过程
  - 4.1 第一阶段：能识别传感器，但没有完整帧
  - 4.2 第二阶段：旧实时预览短暂出图，但反向、卡顿并最终停止
  - 4.3 第三阶段：错误恢复逻辑制造了更多假象
  - 4.4 第四阶段：PID 00/00 是诊断程序自身错误
  - 4.5 Brownout 的正确定位
- 5. 主要根因
  - 5.1 首要原因：实时预览没有复用已验证的采集路径
  - 5.2 首要原因：旧显示方式过于零碎和阻塞
  - 5.3 次要原因：恢复策略破坏了可诊断性
- 6. 为什么现在能够稳定实时显示
- 7. 已排除的错误方向
- 8. 并入主程序时必须遵守的规则
  - 8.1 必须冻结
  - 8.2 推荐任务边界
  - 8.3 上线前验收
- 9. 冻结文件
  - 9.1 纯串口硬件基准
  - 9.2 摄像头 + TFT 日志基准
  - 9.3 稳定实时预览
- 10. 一句话总结

1. 最终结论

本次问题已经通过分层测试闭环。最终确认：

1. **OV2640 摄像头模块正常。**

2. **ESP32-S3 开发板正常。**
3. **自制 PCB 的摄像头与屏幕连接正常。**
4. **冻结引脚表正确，不需要改线：**D7=GPIO43、PCLK=GPIO44、VSYNC=GPIO47、HREF=GPIO46、SDA=GPIO1、SCL=GPIO2、XCLK 不接。
5. **板载 Y1 晶振正常，DVP 并口时序正常。**
6. **3.26V 摄像头供电、RST、PWDN 和 SCCB 上拉均正常。**
7. 之前“短暂出图、画面反、特别卡、随后抓帧失败和 ACK/PID 异常”的主要原因是：**实时预览程序偏离了已经验证的摄像头初始化路径，并且显示与错误恢复策略不合理。**

现在能够稳定显示实时画面，是因为新实现将“已经通过的摄像头采集路径”和“低阻塞的屏幕显示路径”分开处理：

> OV2640 先稳定输出 JPEG QVGA → JPEG 独立解码到 160×120 RGB565 缓冲区 → ST7789 一次整块 SPI 写入。

2. 最终硬件配置

2.1 摄像头引脚

OV2640 信号	ESP32-S3 GPIO	最终状态
D0～D7	16、17、38、39、40、41、42、43	已通过
PCLK	44	已通过
VSYNC	47	已通过
HREF	46	已通过
SDA	1	已通过
SCL	2	已通过
XCLK	不接	使用模组板载 Y1
RST	3.3V	实测约 3.27V
PWDN	GND	实测约 0.02V
3V3	3.3V	实测约 3.26V

SDA、SCL 对 3.3V 的实测电阻约为 6kΩ，说明摄像头模块自身已有上拉，不需要在 PCB 上再并联 4.7kΩ。

2.2 ST7789 引脚

ST7789 信号	ESP32-S3 GPIO
BL	14
CS	10
DC	9
RST	8
MOSI/SDA	11
SCK/SCL	12
VCC	3.3V
GND	GND

3. 已取得的关键证据

3.1 RGB565 独立抓帧通过

早期基准程序在屏幕上得到：

```
PASS 5/5
ACK YES PID 26 VER 42
P7917 V18 H78766
FRAME 160x120 38400B
STATUS: PASS
```

这组结果证明：

- SCCB 可以识别 OV2640；
- PID=0x26、VER=0x42 正确；
- PCLK、VSYNC、HREF 均有活动；
- D0～D7 并口能够形成完整帧；
- RGB565 QQVGA（160×120、38400字节）可以连续抓取。

3.2 纯串口 JPEG 连续抓帧通过

将摄像头单独用杜邦线连接到 ESP32-S3 后，稳定输出：

```
[FRAME] #7  OK 320x240, 8378 bytes, format=4, get=1ms, changed=YES
[FRAME] #8  OK 320x240, 8378 bytes, format=4, get=1ms, changed=YES
[FRAME] #9  OK 320x240, 8317 bytes, format=4, get=1ms, changed=YES
...
[FRAME] #20 OK 320x240, 7686 bytes, format=4, get=1ms, changed=YES
```

`format=4` 为 JPEG。帧长随画面内容在约 7～9KB 间变化，哈希持续变化，说明取到的是不断更新的新画面，而不是重复旧缓冲区。`fb_get` 通常为 0～1ms，说明摄像头 DMA 和帧缓冲工作稳定。

3.3 加入 ST7789 后仍通过

随后在 ESP32-S3 开发板上同时连接 OV2640 与 ST7789，将串口日志同步输出到屏幕，抓帧测试仍然通过。

这一步排除了：

- TFT SPI 接线与摄像头 DVP 引脚冲突；
- ST7789 初始化导致摄像头无法工作；
- 屏幕供电导致持续性摄像头掉电。

3.4 移回自制 PCB 后多次通过

相同的“摄像头 + TFT 日志测试”移回自制 PCB 后重复运行无异常。最终稳定实时预览也在该硬件上正常显示。

因此，自制 PCB 不是本次持续故障的根因。早期出现的 ACK NO、PID FF 和 Brownout 不能再解释为 PCB 固有错误。

4. 故障演变过程

4.1 第一阶段：能识别传感器，但没有完整帧

早期曾出现 PID 正确但 PCLK、VSYNC、HREF 为零，或者初始化成功但 `fb_get` 超时。此时 SCCB 与图像并口属于两个不同层次：

- SCCB 正常，只代表可以读写传感器寄存器；
- 真正抓帧还要求 Y1、PLL、PCLK、VSYNC、HREF、D0～D7 和 LCD_CAM/DMA 全部匹配。

在采用与板载 Y1 相匹配的参数，并执行 JPEG 12/24/20MHz 顺序尝试、RGB565 12MHz 回退后，抓帧恢复。

4.2 第二阶段：旧实时预览短暂出图，但反向、卡顿并最终停止

旧预览曾短暂显示画面，证明采集链路一度已经建立。但显示实现存在两类风险：

1. 逐行或大量小块 SPI 写入，造成明显卡顿；

2. 在摄像头 DMA 和帧缓冲仍工作的同时长时间占用 CPU/显示流程，增大下一帧超时概率。

另外，旧代码在初始化后继续操作 PCLK、VSYNC、HREF 的 GPIO 模式。即使这未必是每一次失败的唯一原因，也属于不必要且高风险的侵入式诊断。稳定版已完全禁止初始化后再对这些 DVP 引脚执行 `pinMode()`。

4.3 第三阶段：错误恢复逻辑制造了更多假象

为了自动恢复，旧预览先后加入过：

- 反复 `esp_camera_deinit()` / `esp_camera_init()`；
- `Wire.end()`；
- GPIO reset；
- 高频 SCCB 重试；
- 失败后覆盖上一次 ACK/PID 状态。

这些处理没有保留“第一次失败现场”，反而可能让后续日志只显示最后一次失败结果。因此出现了：

```
ACK NO
PID FF
ESP_ERR_NOT_SUPPORTED
```

这并不能证明摄像头、PCB或连线已经永久损坏。当回到冻结基准程序后，同一硬件再次通过，说明这些现象主要来自测试状态被反复修改和错误结果被覆盖。

4.4 第四阶段：PID 00/00 是诊断程序自身错误

新的纯串口程序最初出现：

```
ACK=YES, PID=0x00, VER=0x00
```

OV2640 的 `0x0A/0x0B` 身份寄存器位于 Sensor Bank。读取前必须：

```
向寄存器 0xFF 写入 0x01
```

最初测试程序遗漏了 Bank Select，导致总线已经 ACK，但读取的是错误寄存器页。补上 Sensor Bank 选择后，PID/VER 正常恢复为 26/42。

这个错误说明：**诊断代码本身也必须经过验证，不能把每个异常日志直接归咎于硬件。**

4.5 Brownout 的正确定位

过程中确实记录过一次：

```
rst BROWNOUT
```

但后续冷启动显示 `POWERON`，摄像头供电稳定为约 3.26V，并且同一 PCB 能连续抓帧和实时显示。因此：

- Brownout 是当时插拔、启动瞬态或测试过程中的次生事件；
- 它可能加重某一次失败；
- 但它不是最终持续故障的主要原因，也不能用于否定 PCB。

5. 主要根因

5.1 首要原因：实时预览没有复用已验证的采集路径

纯 RGB565 直接初始化曾出现：

```
[WARMUP] FAIL
[FRAME] FAIL: fb_get returned null after 4000ms
```

而使用冻结顺序后，JPEG QVGA 可以连续稳定取帧。因此稳定版采用：

1. JPEG QVGA + PSRAM，PLL参数按 12MHz、24MHz、20MHz 顺序尝试；
2. 仅在 JPEG 路径不可用时，回退到 RGB565 QQVGA + DRAM + 12MHz；
3. 每一种模式初始化后先验证 DVP 活动，再进入抓帧。

关键不是简单地把某个数字改成12MHz，而是保持“已验证的完整初始化顺序、像素格式、帧尺寸、缓冲位置和抓取模式”一致。

5.2 首要原因：旧显示方式过于零碎和阻塞

旧预览把一帧拆成很多次 SPI 操作。即使摄像头已经抓到帧，这种显示方式也会产生：

- 帧率很低；
- 肉眼可见卡顿；
- 长时间持有帧缓冲；
- 摄像头下一帧等待或超时风险上升。

稳定版改为：

1. OV2640 输出 JPEG QVGA；
2. `TJpg_Decoder` 将 JPEG 缩放解码为 160×120 RGB565；
3. 解码结果写入独立的 38400字节缓冲区；
4. ST7789 使用一次 `setAddrWindow` 和一次整块 `writePixels` 完成刷新；
5. 预览循环限制在约12fps，为摄像头 DMA 留出时间。

这也是“现在流畅且不再抓帧超时”的核心原因。

5.3 次要原因：恢复策略破坏了可诊断性

稳定版不再遇错就无限反复初始化。抓帧失败和解码失败分别累计：

```
#帧数  fps  G抓帧失败次数  D解码/显示失败次数
```

这样可以区分：

- `G`增加：摄像头/DMA取帧失败；
- `D`增加：JPEG解码或TFT显示失败；
- 两者均为0：采集与显示链路都正常。

错误现场被保留后，不会再发生“第一次其实成功，后续重试失败却覆盖全部状态”的情况。

6. 为什么现在能够稳定实时显示

当前稳定链路如下：

```
上电稳定
  ↓
释放 UART0 对 GPIO43/44 的占用
  ↓
SCCB 切换 Sensor Bank，确认 PID=26 / VER=42
  ↓
按冻结顺序初始化 JPEG QVGA/PSRAM
  ↓
确认 PCLK、VSYNC、HREF 均有活动
  ↓
丢弃启动阶段前两帧
  ↓
fb_get 获取 JPEG
  ↓
JPEG 缩放解码到独立 160×120 RGB565 缓冲
  ↓
ST7789 单次整块 SPI 刷新
  ↓
归还摄像头 frame buffer
  ↓
循环并统计 FPS、抓帧失败、解码失败
```

这里有三个关键隔离：

1. **采集与显示格式隔离：**摄像头保持稳定 JPEG，不强迫其直接承担屏幕格式。
2. **摄像头帧缓冲与屏幕缓冲隔离：**显示使用独立 RGB565 缓冲，不长时间占用 camera framebuffer。
3. **故障类型隔离：**抓帧失败和解码失败分别统计。

7. 已排除的错误方向

以下项目已通过实测排除，不应再次作为首要怀疑对象：

项目	排除依据
D7=43、PCLK=44 错误	RGB565与JPEG均已连续抓帧
HREF应接GPIO48	实际GPIO46已通过；GPIO48保留板载RGB灯
模块没有晶振	PCLK/VSYNC/HREF有活动且连续出图
摄像头3.3V不足	稳态实测约3.26V并通过
RST/PWDN错误	RST约3.27V、PWDN约0.02V并通过
PCB缺少SCCB上拉	SDA/SCL对3.3V约6kΩ，模块自带上拉
10cm杜邦线太长	20cm曾通过，10cm最终也连续通过
摄像头模块损坏	更换模块后现象相同，随后同一链路通过
自制PCB布线错误	TFT日志测试与实时预览移回PCB后多次通过

8. 并入主程序时必须遵守的规则

8.1 必须冻结

- 不改 D7=43；
- 不改 PCLK=44；
- 不接 XCLK，继续使用板载 Y1；
- 使用 USB CDC 输出日志，初始化前释放 UART0；
- 摄像头优先使用已验证 JPEG QVGA + PSRAM 路径；
- 初始化后不再通过 `pinMode()` 修改PCLK、VSYNC、HREF；
- 不让屏幕逐行、小块、高频阻塞刷新；
- 不在主循环中反复执行 JPEG 编码或摄像头重初始化。

8.2 推荐任务边界

- 摄像头任务只负责取 JPEG 帧；
- 视觉上传任务复制或短暂持有 JPEG 后立即归还 framebuffer；
- 屏幕预览任务使用独立解码缓冲；
- 主对话任务不得等待摄像头解码或SPI刷新；
- 自动重试必须有限次数、带退避，并保留第一次错误码。

8.3 上线前验收

1. 冷启动10次，PID/VER识别成功率100%；
2. 连续抓取至少500帧，`G=0`；
3. 连续预览10分钟，`D=0`且无冻结；
4. 同时运行按键扫描，确认唤醒/打断无明显延迟；
5. 再开启Wi-Fi和视觉上传，观察PSRAM、堆内存和帧率；
6. 最后加入语音播放，检查SPI、DMA和任务优先级是否相互阻塞。

9. 冻结文件

9.1 纯串口硬件基准

```
C:\Users\xijie\Documents\Codex\2026-09-05\longlinshuoxi-wav\
ov2640_serial_only_test\ov2640_serial_only_test.ino
```

SHA-256：

```
0A1CB77F771F96D09FE58C9D8079D26BC5E22EF6900E2691F1EB100F3BC1C502
```

9.2 摄像头 + TFT 日志基准

```
C:\Users\xijie\Documents\Codex\2026-09-05\longlinshuoxi-wav\
ov2640_tft_serial_test\ov2640_tft_serial_test.ino
```

SHA-256：

```
5DF4A06A6071A09F649295EAF847D4C76B5CAB81B32E80289DF55C1B3054975E
```

9.3 稳定实时预览

```
C:\Users\xijie\Documents\Codex\2026-09-05\longlinshuoxi-wav\
ov2640_live_preview_stable\ov2640_live_preview_stable.ino
```

SHA-256：

```
BCCD61A7A56EDAF899CB8A265C688BA4B3B2EABF2B3DAA5E6E524363558E8F7C
```

10. 一句话总结

> 硬件一直具备工作条件；真正让实时画面从“偶发、卡顿、失联”变成稳定运行的关键，是回到已验证的 JPEG 摄像头初始化路径，正确读取 OV2640 寄存器页，停止侵入式和无限重试式诊断，并把 JPEG 解码与 ST7789 刷屏改成独立缓冲加单次整块 SPI 传输。